Vector-Quantized World Model: Latent Feedback Motion Planning for Robot Manipulation

Xuezhou Xu*[1], Chung-Pang Wang*[1], Zhenzhen Li[2], and Benjamin Riviere[1]

***Abstract*—Visual world models have shown strong potential for robot planning, yet long-horizon manipulation remains challenging due to compounding prediction errors, large search complexity, and latent plans that the low-level controller cannot reliably reach. In this work, we introduce Vector-Quantized World Model (VQ-WM), a latent feedback motion planning framework that integrates discrete long-horizon search with continuous visual feedback control: a discrete latent predictor models transitions between vector-quantized latent states, allowing the planner to look far ahead and find a shortest path to the goal through discrete search and a continuous learned controller sequentially reaches the planned states through visual feedback, correcting deviations to mitigate compounding errors. Whereas most vector quantization uses Euclidean distance, we use a novel estimator of the first-passage-time as a quasi-metric, which aligns the planning abstraction with the controller's capabilities. We evaluate on the physical robot manipulation of the Tower of Hanoi puzzle: whereas existing state-of-the-art foundation models (VLAs, WAMs) only succeed when the solution trace in their training dataset, our method achieves 100% success rate with only short-play data, demonstrate its ability to compose long-horizon solutions beyond the short demonstrated trajectories and execute them reliably. Our solution also outperforms strongest world-models by 13.3–85.3 percentage points on simulation benchmarks spanning manipulation and locomotion, demonstrating consistent progress and new capabilities.**

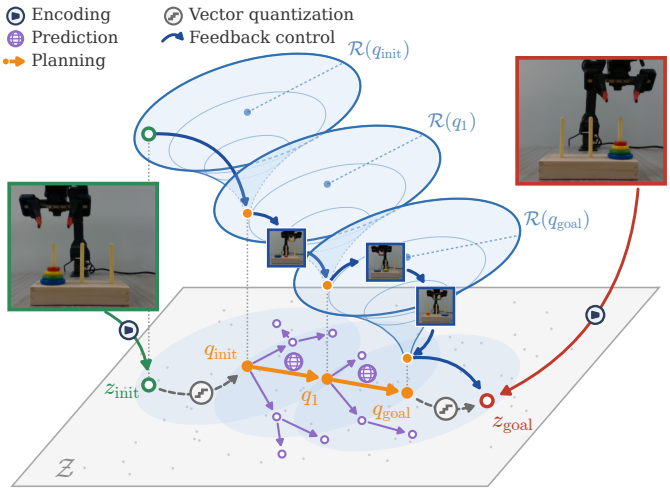


Fig. 1: Overview of VQ-WM. The initial (green) and goal (red) images are encoded into latent states and vector-quantized according to controller reachability, so that each vector-quantized state serves as a target the controller is expected to reach. The planner performs discrete search over these vector-quantized states using a discrete latent predictor. A continuous controller then drives the robot toward each planned target in turn, using visual feedback to correct deviations along the way.

I. Introduction

Visual world models combine latent representations with predictive dynamics, enabling planners to evaluate action sequences directly from visual inputs and select those that lead toward a desired long-term outcome before acting in the environment. Recent advances in model architectures and training methods allow these models to learn from reward-free offline trajectories and perform zero-shot planning from visual observations [1], [2]. However, long-horizon manipulation tasks requiring multi-stage decision making and precise sequential execution remain challenging. Existing world-model planners typically optimize action sequences through autoregressive rollouts in continuous latent space. As the planning horizon grows, prediction errors can accumulate across rollout steps, while longer action sequences become harder to optimize. These limitations constrain the effective planning horizon and can cause failure even when the dynamics model is accurate over short horizons.

Recent work has improved long-horizon world-model planning by addressing these limitations. Hierarchical approaches such as H-JEPA learn multiple levels of latent abstraction to support coarse-to-fine planning [3]. Discrete latent representations have also been explored for hierarchical planning and control [4], [5], while Hi-LeWM investigates vector-quantized macro-actions for high-level prediction and planning [6]. Hydra [7] uses vector-quantized multimodal intents for sampling-based navigation planning. Other approaches improve the geometry of the latent planning space by encouraging locally straighter latent trajectories [8]. Despite these advances, plans formed in latent space may still be difficult to execute reliably. Replanning can correct deviations, but does not by itself ensure that proposed targets are compatible with the controller's capabilities. The planning abstraction should reflect which states the controller can reliably reach and stabilize, a principle central to classical *feedback motion planning* [9], [10].

The LQR-Trees framework makes this connection explicit by associating each trajectory with a feedback controller and a region of attraction, characterizing the states from which the controller can recover deviations and steer the system toward the goal [11].

Motivated by these limitations and the principle of coupling planning and control, we propose *Vector-Quantized World Model* (VQ-WM), a *latent feedback motion planning* frame-

*These authors contributed equally to this paper.
[1]New York University, New York, NY, USA. [2]NVIDIA.

work for long-horizon robotic manipulation. As illustrated in Fig. 1, VQ-WM uses vector quantization to construct a discrete planning space whose vector-quantized states serve as targets for visual feedback control. The planner runs discrete search on the resulting graph to find a path to the goal, and the controller runs tracking control between consecutive vector-quantized states on the path. The central design choice is to define vector quantization through the controller's ability to reach these targets, aligning the planning abstraction with the controller's capabilities. In particular, we use the first-passage time, which is the number of controller calls until a trajectory driven by the controller first enters a neighborhood of its target. We estimate this quantity through closed-loop controller rollouts in a latent world model learned from offline data, and use it to construct the state codebook and assign continuous latent states to vector-quantized states.

Our main contributions are:

- We introduce VQ-WM, a latent feedback motion planning framework that uses vector quantization to integrate discrete long-horizon search with continuous visual feedback control.
- We propose first-passage time as a measure of controller reachability for state vector quantization and estimate it through closed-loop controller rollouts in a learned latent world model. The first-passage time from one state to another is the number of controller calls until the closed-loop trajectory first reaches it. This aligns the planning abstraction with the controller's capabilities and supports effective planning with compact state codebooks.
- We validate VQ-WM in hardware and simulation. Learning from short play data without full-task demonstrations, it solves the physical Tower of Hanoi puzzle with 100% task progress, outperforming the competitive industrial baselines by at least 95 percentage points. In simulation, it surpasses the strongest world-model planner, H-JEPA, by 13.3–34.6 percentage points on challenging long-horizon manipulation tasks, and adding proprioceptive inputs to the controller raises Walker success from 22.0% to 97.3%.

II. Related Work

A. World Models for Visual Planning

World models learn predictive dynamics from observations to support action selection. Early approaches combine visual representations with latent dynamics for control [12], [13], or predict future images for goal-directed manipulation [14]. Universal Planning Networks jointly learn representations and differentiable action planning [15], while Dreamer learns behavior through latent imagination [16] and TD-MPC methods combine short-horizon latent planning with value estimation [17], [18]. More recent methods support image-goal planning from pretrained visual features or reward-free offline trajectories [1], [2], [19], [20]. However, continuous action-sequence planning becomes increasingly challenging over long horizons as search complexity grows rapidly and prediction errors can accumulate during autoregressive rollouts. VQ-WM instead performs discrete search over vector-quantized states without optimizing the full sequence of low-level actions.

B. Abstractions for Long-Horizon Planning

Temporal hierarchies decompose extended tasks into shorter planning problems [4]. HWM learns world models at multiple temporal scales and demonstrates hierarchical planning for real-world pick-and-place from a single goal image [21]. H-JEPA combines multiple prediction horizons with latent spaces at different abstraction levels [3]. However, predicted subgoals can still be difficult for the execution mechanism to realize [6]. Vector quantization provides a compact state space for efficient discrete search, but the resulting state assignments must also reflect controller reachability. Graph-based methods incorporate reachability into planning through learned goal-conditioned control. SoRB searches over replay-buffer states using distances learned through goal-conditioned reinforcement learning [22], while L3P learns latent landmarks and graph edges from reachability estimates distilled from value functions [23]. VQ-WM vector-quantizes states based on first-passage times estimated through closed-loop controller rollouts in a learned world model, then learns vector-quantized transitions from offline trajectories for discrete search. This couples the planning abstraction to the controller's ability to reach the assigned targets.

C. Feedback Motion Planning and Goal-Conditioned Control

Feedback motion planning accounts for feedback control during planning to avoid trajectories that are difficult or costly to stabilize [11]. Classical approaches compose local controllers so that each brings the system into a region where the next can succeed, using domains of attraction or trajectory funnels to characterize these regions [9]–[11]. Goal-conditioned methods learn reusable behaviors from play [24] or generate goal-directed action sequences through inverse dynamics models [25]. Composing such behaviors into long-horizon plans requires accounting for the states from which each target can be reached. VQ-WM applies this composition principle to a learned goal-conditioned controller in visual latent space. Instead of formally verifying funnels, it estimates controller reachability using first-passage times from closed-loop rollouts in a learned world model. These estimates guide the composition of local behaviors into long-horizon plans, which the controller executes using continuous visual feedback. We demonstrate reliable long-horizon manipulation on real hardware using this latent feedback motion planning framework.

III. Problem Definition and Background

Our work follows a vision-based control framework, which models the environment as a controlled Markov process $\langle \mathcal{O}, \mathcal{A}, P_O \rangle$, where $\mathcal{O}$ is the observation space and $\mathcal{A}$ is the physical action space. At each time step t, the robot receives an observation $o_t \in \mathcal{O}$ and applies an action $a_t \in \mathcal{A}$. The transition distribution $P_O(o_{t+1} \mid o_t, a_t)$ describes how the robot's observation changes in response to its action.

Given a pre-collected, reward-free offline dataset $\mathcal{D} = \{(o_0^{(n)}, a_0^{(n)}, \ldots, a_{T_n-1}^{(n)}, o_{T_n}^{(n)})\}_{n=1}^{N}$ of N observation–action trajectories with respective episode lengths T_n, our goal at test time is to steer the system from an initial observation o_{init} to a state specified by a goal image o_{goal}.

IV. METHODS

A. Overview

At a high level, VQ-WM interweaves discrete planning over a controller-aware state codebook $\mathcal{Q}$ with continuous feedback control using π.

The components of VQ-WM are $\{E_Z, E_U, P_Z, P_Q, \phi_Z, \phi_U, \pi\}$. The pretrained, frozen encoder E_Z maps observations to continuous latent states $z_t = E_Z(o_t) \in \mathcal{Z}$. For example, in this work, we use V-JEPA 2 [20]. The trained encoder E_U maps pairs of vector-quantized states to continuous latent actions. The quantizers ϕ_Z and ϕ_U assign these representations to entries in finite state and action codebooks $\mathcal{Q} \subset \mathcal{Z}$ and $\mathcal{U}$, respectively. The predictors P_Z and P_Q model continuous and vector-quantized latent transitions, respectively, and the goal-conditioned controller $\pi(\cdot \mid z, z')$ maps the current and target latent states to a physical action.

A natural choice for the state quantizer ϕ_Z is to assign each latent state to its nearest codebook entry under Euclidean distance. However, geometric proximity in latent space does not necessarily reflect the effort required by a feedback controller to reach a target. We therefore use *first-passage time* (FPT) as a directed, controller-dependent reachability cost for state vector quantization. FPT measures the number of controller calls required for a closed-loop trajectory to first enter a neighborhood of its target. Using such a distance measure for state vector quantization couples discrete planning with continuous control, enabling efficient search over targets aligned with the controller's capabilities.

During offline learning, we first train π and the continuous latent predictor P_Z, which predicts the next latent state as $\hat{z}_{t+1} \sim P_Z(\cdot \mid z_t, a_t)$. Their closed-loop rollouts provide first-passage time estimates $d(z, z')$, which we use to construct $\mathcal{Q}$ and quantize continuous latent states $z \in \mathcal{Z}$ as $q = \phi_Z(z) \in \mathcal{Q}$. With $\mathcal{Q}$ fixed, we jointly learn the latent action encoder E_U, action codebook $\mathcal{U}$, and discrete latent predictor P_Q. The predictor P_Q outputs a categorical distribution $P_Q(q' \mid q, u)$ over the next vector-quantized state, conditioned on the current vector-quantized state q and vector-quantized action u. At inference time, our planner performs graph search using P_Q to select a sequence of intermediate targets in $\mathcal{Q}$, which π tracks using visual feedback. The inference and training pipelines of VQ-WM are shown in Fig. 2.

B. Inference

VQ-WM performs discrete planning and continuous feedback control at different rates, as shown in Fig. 2(a). Planning operates at a slower rate and uses the discrete latent predictor P_Q to expand the search graph online, while the goal-conditioned controller π operates at a faster rate with visual feedback.

1) Planning Loop: Our planner searches the vector-quantized state graph for a shortest path to the goal. Given initial and goal observations, we encode them as $z_{\text{init}} = E_Z(o_{\text{init}})$ and $z_{\text{goal}} = E_Z(o_{\text{goal}})$, then quantize them into the vector-quantized states q_{init} and q_{goal}.

Starting from q_{init}, our planner explores successor states predicted by $P_Q(\cdot \mid q, u)$ for all vector-quantized actions $u \in \mathcal{U}$ at each expanded state q. It builds the graph online by continuing to expand newly discovered vector-quantized states until q_{goal} is found. The predicted categorical distributions define a stochastic transition graph that can be used with stochastic shortest-path algorithms. In our implementation, we retain the most likely successor $\arg\max_{q' \in \mathcal{Q}} P_Q(q' \mid q, u)$ for each state–action pair and use breadth-first search (BFS) to find a shortest path from q_{init} to q_{goal} with the fewest graph transitions. We denote this path by the sequence of vector-quantized states $(q_0, q_1, \ldots, q_L)$, where $q_0 = q_{\text{init}}$ and $q_L = q_{\text{goal}}$. If the goal is predicted to be reachable within R controller calls, $d(z_{\text{init}}, z_{\text{goal}}) \leq R$, we bypass graph search and use π with z_{goal} as its target.

Our planner can be implemented in a receding-horizon manner by periodically updating the plan from the current observation and executing only its initial portion before replanning, or in open-loop by setting the replanning frequency to zero. The quantization procedure makes planning computationally efficient, allowing VQ-WM to compute a complete plan spanning approximately five minutes of robot execution in under one second on an NVIDIA RTX 5080 GPU. In our experiments, to evaluate the stability of our feedback motion planning framework, we use open-loop planning once per episode and execute the resulting plan using continuous visual feedback.

2) Control Loop: The continuous controller operates at a higher rate than the planner, tracking the planned path using visual feedback. At each control step, we encode the current observation as $z_t = E_Z(o_t)$ and apply an action $a_t \sim \pi(\cdot \mid z_t, q_i)$ toward the current planned state q_i. The controller follows the planned states in order, beginning with q_{init}, and finally target z_{goal}. For each planned transition, it runs for the number of controller calls specified by the first-passage time estimated between consecutive planned states before switching to the next target. Execution succeeds when the task is completed and fails if it remains incomplete after the final target has been attempted.

C. Training

We train VQ-WM from the offline dataset $\mathcal{D}$ defined in Sec. III, following the pipeline in Fig. 2(b). We first train the goal-conditioned controller π and continuous latent predictor P_Z, then construct the state quantizer ϕ_Z and codebook $\mathcal{Q}$, and finally learn the vector-quantized latent dynamics. A pretrained latent state encoder E_Z maps observations to latent states $z_t = E_Z(o_t)$ and remains frozen throughout training.

1) Controller: We train the goal-conditioned controller π on state–goal pairs (z_t, z_{t+r}) from the same offline trajectory, sampling $r \sim \text{Uniform}\{0, \ldots, R\}$ to provide targets at different temporal distances. When $r = 0$, we replace a_t with a hold action so the controller learns to maintain the current state. We train π by minimizing:

$$\mathcal{L}_{\text{control}} = \left\| a_t - \hat{a}_t \right\|_1, \quad \hat{a}_t = \mathbb{E}_{a \sim \pi(\cdot \mid z_t, z_{t+r})}[a]. \tag{1}$$

For a target z', we approximate its R-step region of attraction $\mathcal{R}(z')$ as the set of initial latent states from which closed-loop control with π reaches an ϵ-neighborhood of z' within R controller calls with arrival tolerance $\epsilon > 0$.

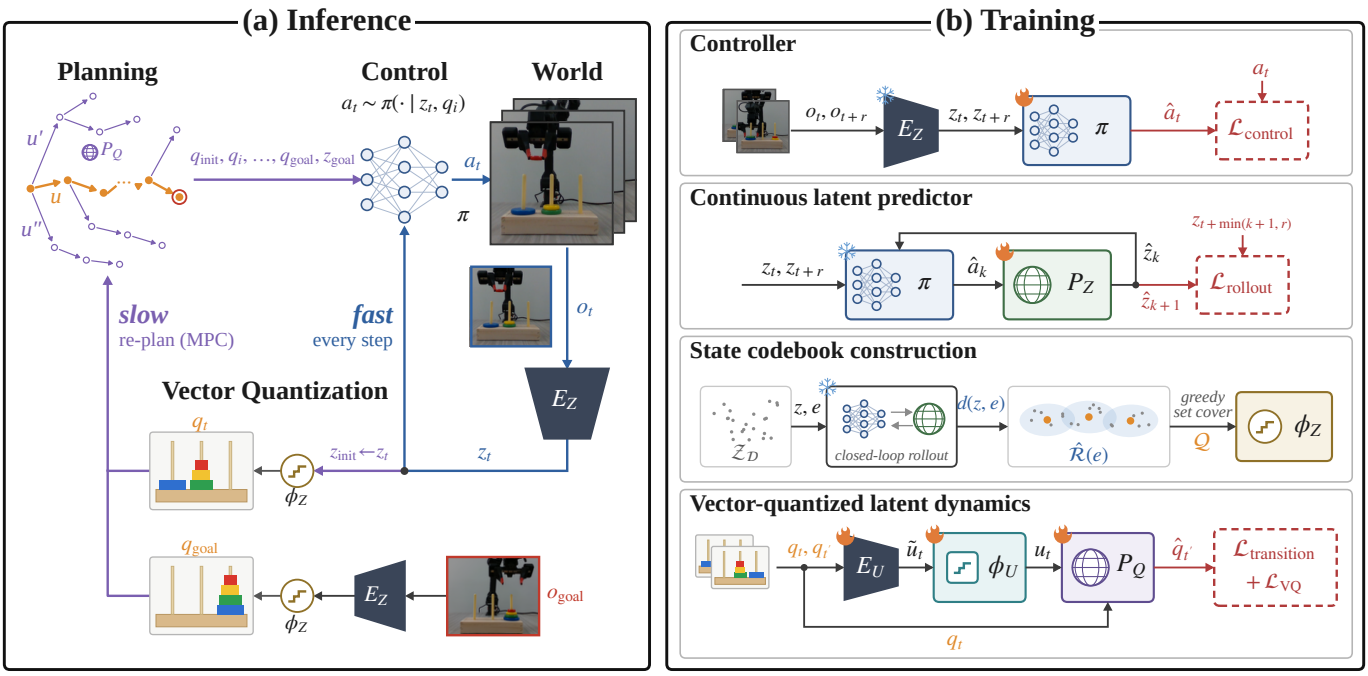


Fig. 2: VQ-WM components and pipeline. (a) Inference encodes the initial and goal images, searches over state codes, and executes the planned targets with visual feedback. (b) Offline training learns the controller and continuous latent predictor, performs state vector quantization using first-passage time, and then learns vector-quantized latent dynamics.

Algorithm 1 Codebook Construction via Greedy Set Cover

Require: Latent states $\mathcal{Z}_\mathcal{D}$, first-passage time estimate d, controller-call budget R, number of neighbors K
Ensure: State codebook $\mathcal{Q}$, vector-quantized states $\{q\}_{z \in \mathcal{Z}_\mathcal{D}}$

for each $e \in \mathcal{Z}_\mathcal{D}$ **do** ▷ approximate regions of attraction
  $\hat{\mathcal{R}}(e) \leftarrow \{e\}$
  for each $z_{n,t} \in \mathrm{KNN}(e, K)$ with $\rho = d(z_{n,t}, e) \le R$ **do**
    $\hat{\mathcal{R}}(e) \leftarrow \hat{\mathcal{R}}(e) \cup \{z_{n,t-m}\}_{m=0}^{\min(R-\rho,t)}$
  end for
end for
$\mathcal{Q} \leftarrow \emptyset$, $\mathcal{V} \leftarrow \mathcal{Z}_\mathcal{D}$ ▷ greedy set cover
while $\mathcal{V} \neq \emptyset$ **do**
  $e^\star \leftarrow \arg\max_{e \in \mathcal{Z}_\mathcal{D}} |\hat{\mathcal{R}}(e) \cap \mathcal{V}|$
  $\mathcal{Q} \leftarrow \mathcal{Q} \cup \{e^\star\}$, $\mathcal{V} \leftarrow \mathcal{V} \setminus \hat{\mathcal{R}}(e^\star)$
end while
assign each $z \in \mathcal{Z}_\mathcal{D}$ to $q = \phi_Z(z)$
return $\mathcal{Q}$, $\{q\}_{z \in \mathcal{Z}_\mathcal{D}}$

2) Continuous Latent Predictor: The continuous latent predictor P_Z predicts the next latent state from the current latent state and a physical action. We supervise its predictions over closed-loop rollouts with π, matching the rollout procedure used for first-passage time estimation. Given a latent state z_t and a target $z' = z_{t+r}$ with $r \sim \mathrm{Uniform}\{0, \ldots, R\}$, we initialize $\hat{z}_0 = z_t$ and predict R steps with the closed-loop rollout. The predicted trajectory is supervised with the recorded one, which is held at the target once it is reached:

$$\mathcal{L}_{\text{rollout}} = \frac{1}{R} \sum_{k=0}^{R-1} \left\| \hat{z}_{k+1} - z_{t+\min\{k+1, r\}} \right\|_2^2. \tag{2}$$

3) State Vector Quantization: We estimate the first-passage time from z to z' by rolling out the controller π toward z' in closed loop with the continuous latent predictor P_Z:

$$\hat{a}_k \sim \pi(\cdot \mid \hat{z}_k, z'), \quad \hat{z}_{k+1} \sim P_Z\big(\cdot \mid \hat{z}_k, \hat{a}_k\big), \quad \hat{z}_0 = z, \\ d(z, z') = \min\big\{k \in \{0, \ldots, R\} : \|\hat{z}_k - z'\|_2 \le \epsilon\big\}. \tag{3}$$

We set $d(z, z') = \infty$ if the target is not reached within R calls. This first-passage time estimate guides construction of the state codebook. During inference, it is used to vector quantize the initial and goal states and determine the number of controller calls allocated to each planned target.

Let $\mathcal{Z}_\mathcal{D} \subseteq \mathcal{Z}$ denote the latent states encoded from the dataset, with $z_t^{(n)} = E_Z(o_t^{(n)})$ for step t of trajectory n. For each candidate code $e \in \mathcal{Z}_\mathcal{D}$, we identify states z from which π can reach e within R calls, using $d(z, e)$:

$$\hat{\mathcal{R}}(e) = \{z \in \mathcal{Z}_\mathcal{D} : d(z, e) \le R\}. \tag{4}$$

This set estimates the R-step region of attraction of e. We seek the smallest codebook whose estimated regions of attraction collectively cover $\mathcal{Z}_\mathcal{D}$, and use greedy set cover to approximate this selection (Fig. 2(b)).

For efficient computation, we approximate $\hat{\mathcal{R}}(e)$ by using local rollout evaluations and recorded trajectories. Starting with all states uncovered, greedy set cover repeatedly adds the code covering the largest number of uncovered states and marks these states as covered. The procedure stops when the selected sets cover the entire encoded dataset, $\mathcal{Z}_\mathcal{D} = \bigcup_{q \in \mathcal{Q}} \hat{\mathcal{R}}(q)$. The vector quantizer then assigns each state to the code it reaches in the fewest controller calls:

$$\phi_Z(z) = \arg\min_{q \in \mathcal{Q}} d(z, q). \tag{5}$$

Since d is directed, we assign the goal z_{goal} to the code from which the controller can reach it in the fewest calls, $q_{goal} = \arg\min_{q \in \mathcal{Q}} d(q, z_{goal})$. Algorithm 1 details region approximation, greedy selection, and state assignment.

4) Vector-Quantized Latent Dynamics: With the state codebook fixed, we jointly learn the latent action encoder, the action codebook and the discrete latent predictor (Fig. 2(b)). The fixed state codebook $\mathcal{Q}$ consists of latent states selected from the encoded dataset $\mathcal{Z}_{\mathcal{D}}$, while $\mathcal{U}$ denotes the learned action codebook. Let $q_t = \phi_Z(z_t)$ denote the vector-quantized state of z_t. The vector-quantized action u_t represents a transition between vector-quantized states, whereas a_t is the physical action produced by the controller π. Let $t' = \min\{s > t : q_s \neq q_t\}$ denote the first subsequent time step at which the vector-quantized state changes. We train on the resulting transitions $(q_t, q_{t'})$. Given such a pair, E_U produces a continuous latent action $\tilde{u}_t = E_U(q_t, q_{t'})$, which ϕ_U assigns to its nearest entry in the action codebook, $u_t = \phi_U(\tilde{u}_t) = \arg\min_{u \in \mathcal{U}} \|\tilde{u}_t - u\|_2^2$. We train the action codebook with the standard vector quantization objective [26],

$$\mathcal{L}_{\text{VQ}} = \|\text{sg}(\tilde{u}_t) - u_t\|_2^2 + \beta\|\tilde{u}_t - \text{sg}(u_t)\|_2^2, \tag{6}$$

where sg stops gradients and β weights the commitment loss. The discrete latent predictor P_Q outputs a categorical distribution over the vector-quantized states in $\mathcal{Q}$, conditioned on the current vector-quantized state and vector-quantized action, $P_Q(q_{t'} \mid q_t, u_t)$. It is trained with the cross-entropy loss $\mathcal{L}_{\text{transition}} = -\log P_Q\big(q_{t'} \mid q_t, u_t\big)$, and the joint training objective is $\mathcal{L}_{\text{transition}} + \mathcal{L}_{\text{VQ}}$.

V. Experiments

We evaluate VQ-WM through real-world and simulated experiments to investigate three questions:

- **Q1: Compositionality.** Can VQ-WM compose local transitions into long-horizon solutions beyond demonstrated trajectory lengths?
- **Q2: Long-horizon execution.** Can discrete planning with continuous feedback control sustain performance across long-horizon manipulation tasks?
- **Q3: Controller-aware abstraction.** Does first-passage-time-based vector quantization provide a more compact and effective planning abstraction than Euclidean vector quantization?

We investigate Q1 and Q2 through real-world Tower of Hanoi experiments and simulated manipulation benchmarks, and Q3 through controlled vector quantization ablations in simulated Tower of Hanoi. We additionally examine the flexibility of separating planning and control by incorporating proprioceptive observations in the Walker locomotion task.

A. Simulation Benchmarks

Long-horizon manipulation. We evaluate manipulation in the Cube and Scene environments from Stable-WorldModel [27], using task-specific evaluation protocols. Cube-Easy follows LeWM's protocol [2], with a goal 25 environment steps ahead and an execution budget of 50 steps. Cube-Hard follows H-JEPA's protocol [3], using 50 held-out, 20-step windows centered on a grasp and an execution budget of 30 steps to evaluate complete pick-and-place behavior. For Scene, we define an evaluation with initial and goal images sampled 50 environment steps apart from expert trajectories and an execution budget of 100 steps. Reaching the goal typically requires sequential subtasks involving the cube, buttons, drawer, or window, with success requiring all objects to simultaneously match their goal states.

VQ-WM achieves the highest success on all three tasks (Table I), and its margin over the strongest baseline grows with the task horizon: 8.0 percentage points on Cube-Easy, 13.3 on Cube-Hard, and 28.0 on Scene. We attribute these gains to two mechanisms. First, VQ-WM decouples representation from planning geometry. Temporal straightening (TS) [8] couples the two by reshaping the latent space so that Euclidean distance becomes a useful planning metric, whereas VQ-WM keeps a frozen encoder and learns the geometry separately through first-passage time under the controller. VQ-WM improves over TS by 16.7 and 28.0 percentage points on Cube-Easy and Scene, and reaches 73.3% on Cube-Hard, where TS largely fails at 10.7%. Second, VQ-WM replaces long-horizon action optimization with discrete search over controller-reachable vector-quantized states. Cube-Easy goals often involve little change in cube position, so short-horizon planners already perform well, whereas Scene requires sequential subtasks over which errors in continuous rollouts accumulate. Compared with the predicted subgoals of H-JEPA [3], which the controller may fail to reach, planning over reachable vector-quantized states is consistent with the 13.3-point gain on Cube-Hard over this strongest hierarchical baseline.

TABLE I: Long-horizon Manipulation success rates.

Method	Cube-Easy	Cube-Hard	Scene
LeWM [2]	74.0 ± 2.8	32.0 ± 6.1	42.7 ± 1.9
TS [8]	68.0 ± 2.8	10.7 ± 0.9	47.3 ± 2.5
PLDM [19]	62.0 ± 4.3	11.3 ± 3.8	44.7 ± 4.1
H-JEPA [3]	76.7 ± 2.5	60.0 ± 5.0	40.7 ± 2.5
Ours	$\mathbf{84.7 \pm 2.5}$	$\mathbf{73.3 \pm 0.9}$	$\mathbf{75.3 \pm 5.0}$

Locomotion. We evaluate goal-conditioned motion control in the DeepMind Control Walker environment from Stable-WorldModel [27] to examine the advantage of separating planning from control. Goal images are sampled 50 steps ahead on expert walking trajectories, and success requires the torso height, torso pitch, and all six leg joint angles to be within 0.3 of their goal values. With visual inputs only, both DINO-WM [1] and VQ-WM perform poorly, reaching 12.7% and 22.0%, since such poses depend on precise joint coordination that images convey poorly. After adding proprioception, DINO-WM remains at 12.0%, whereas VQ-WM rises to 97.3% (Table II). The difference arises because VQ-WM handles control separately and its vector quantization is controller-aware: the additional input improves the controller, and the vector-quantized states adapt to what it can now reach without any change to the planner. The same property allows VQ-WM to be readily extended to other sensing modalities at

TABLE II: Success rate (%) on Walker with different observation inputs.

Method	Images	Images + Proprio.
DINO-WM	12.7 ± 3.4	12.0 ± 4.9
Ours	**22.0±5.9**	**97.3±2.5**

the control level.

B. Real-World Experiments

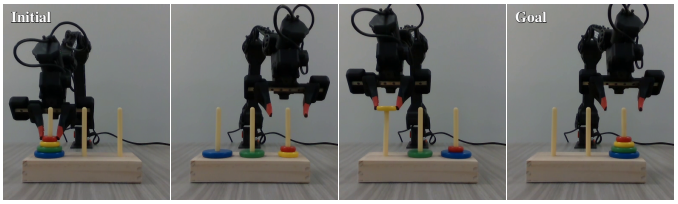


Fig. 3: Real-world execution of VQ-WM on the four-disk Tower of Hanoi puzzle.

To evaluate VQ-WM's ability to compose and execute long-horizon plans from local behaviors, we test it on a real-world four-disk Tower of Hanoi puzzle using a WidowX AI robotic arm [28], as shown in Fig. 3. The goal is to transfer the entire stack to another rod, moving one disk at a time without placing a larger disk on a smaller one. Solving the full puzzle optimally requires 15 puzzle moves (H15), combining planning over minutes of execution with precise grasping, alignment, and disk insertion. We also evaluate single-move tasks (H1) to compare performance across short and long task horizons.

Data Collection. Tower of Hanoi play data was collected autonomously using a WidowX AI arm at 30 Hz. A symbolic planner generated randomized legal move sequences using board states detected by the video object segmentation model Cutie [29] at episode boundaries. Each disk transfer was executed using a predefined Cartesian pick-and-place motion.

Evaluation Protocol. We compare VQ-WM against the world action model (WAM) Cosmos Policy [30], the vision-language-action model (VLA) $\pi_{0.5}$ [31], and the world foundation model V-JEPA 2-AC [20] under expert training on H1 and H15 and non-expert training on H15. The expert dataset contains complete solution demonstrations for six initial–goal configuration pairs. The VLA and WAM baselines are trained jointly on these six tasks. In the non-expert dataset setting, the VLA and WAM baselines are trained on play-data subtasks spanning at most five puzzle moves and evaluated on full H15 tasks, testing whether they can compose shorter subtasks into long-horizon solutions. To reduce unsupported predictions during planning, we augment VQ-WM's discrete latent predictor with an additional rejection class for unseen transitions and exclude predictions in this class from graph search. For V-JEPA 2-AC, we restrict the action sampler to actions observed in the dataset. Following [20], we also evaluate V-JEPA 2-AC* with six manually specified subgoal images per puzzle move. These subgoals provide the high-level solution structure, leaving the model to perform local planning and control between successive targets. We evaluate each method on the same six test cases with three trials per case, using a single NVIDIA RTX 5080 GPU for inference. We present the results in Fig. 4. The Tower of Hanoi puzzle challenges both planning and control: solving it requires planning far ahead while repeatedly executing precise transfers of small disks between pegs.

Expert data setting. Cosmos Policy and $\pi_{0.5}$ both achieve 100% task progress on H1, but drop to 76% and 23%, respectively, on H15. This decline suggests that execution errors compound across successive disk transfers, reducing reliability over the full puzzle. V-JEPA 2-AC uses the cross-entropy method (CEM) [32] to optimize continuous action sequences over a fixed horizon, executing only the first action before replanning, as in receding-horizon control. Extending its lookahead requires searching over longer action sequences, whose predicted outcomes become less reliable as prediction errors accumulate across autoregressive latent rollouts. Frequent replanning alone cannot ensure progress when the benefits of necessary intermediate actions lie beyond this horizon. This is particularly relevant to Tower of Hanoi, where intermediate puzzle moves and physical grasping and lifting motions may not immediately bring the observed state closer to the final goal. V-JEPA 2-AC achieves 0% task progress on both H1 and H15, suggesting that even a single puzzle move lies beyond its effective planning horizon. By providing subgoal images manually selected from the dataset as sequential planning targets, V-JEPA 2-AC* raises its task progress to $94\%\pm6\%$. The gap between the two settings points to a limitation: the model can reach a nearby target when one is provided, but cannot discover the intermediate moves on its own. Even with manually provided subgoals, V-JEPA 2-AC* struggles to execute full H15 solutions reliably. VQ-WM instead searches for the shortest path through controller-aware latent abstractions, allowing the planner to reason far into the future without optimizing the full sequence of low-level actions. This also makes our planner efficient, allowing us to use an infinite replanning interval and compute the complete solution in a single planning call. A continuous controller then executes the plan by sequentially reaching the targets using visual feedback to correct local deviations and mitigate compounding execution errors. VQ-WM achieves 99.6% task progress on H15, demonstrating reliable execution over the full 15-move horizon.

Non-Expert play data setting. Cosmos Policy and $\pi_{0.5}$ achieve only 5% and 3% task progress, respectively, indicating that these behavior-cloning policies struggle to compose shorter subtasks into complete H15 solutions. Although V-JEPA 2-AC can plan beyond demonstrated trajectories, it still struggles with long-horizon planning and execution, as discussed above. Providing manual subgoals improves task progress to 27% for V-JEPA 2-AC*, though reliable execution over the full H15 horizon remains challenging. VQ-WM achieves 100% task progress, demonstrating its ability to compose observed transitions into long-horizon solutions and reliably execute the resulting plans.

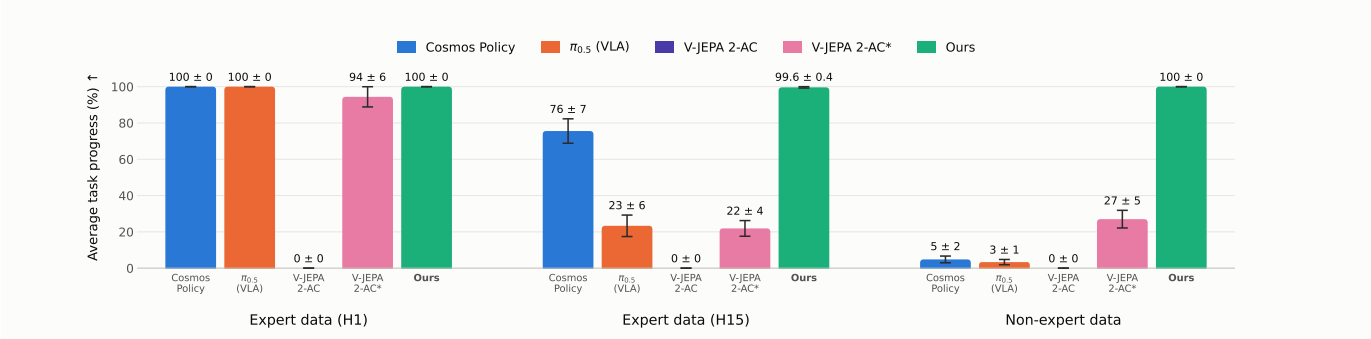


Fig. 4: **Real-world results.** VLA and WAM baselines without runtime search fail to extrapolate from short play subtasks to full-puzzle solutions, whereas VQ-WM composes local behaviors into complete plans and executes them reliably. V-JEPA 2-AC struggles even on H1, suggesting a limited effective planning horizon. V-JEPA 2-AC* receives manually specified subgoals that supply the solution sequence, leaving the model to perform local planning and control.

C. Ablation on State Vector Quantization

To enable extensive trials for statistically meaningful comparisons, we conduct this ablation study on a simulated version of Tower of Hanoi in MuJoCo [33] with the same robotic arm and puzzle setup as our real-world experiment. The dataset contains 104 walk trajectories covering all legal disk-move transitions. We compare first-passage time with squared Euclidean distance for codebook construction and state assignment, keeping the encoder, controller, and training procedure unchanged. Each point is evaluated over three seeds with 50 trajectories per seed, using randomly sampled initial and goal configurations spanning shortest-path lengths of 1–15 disk moves.

Figure 5 shows that first-passage time achieves approximately 90% peak success, compared with 40% for Euclidean distance, an improvement of about 50 percentage points. It also sustains high success across a broad range of codebook sizes, roughly 250–2000, spanning an eightfold change. Performance decreases at both extremes: overly small codebooks merge distinct puzzle configurations, producing unexecutable plans, while overly large codebooks leave fewer observed examples per transition, reducing the accuracy of the discrete latent predictor.

These results highlight the benefits of using first-passage time to align state abstraction and feedback control with the controller's capabilities. First, Euclidean distance ignores controller reachability, so consecutive planned targets may be unreachable under the controller. Second, during control loop, the Euclidean variant declares a target reached when $\|z_t - q_i\|_2 \leq \epsilon$. Since latent proximity does not always indicate physical arrival, the target may switch too early or fail to switch after arrival. Instead, in the first-passage-time variant, the number of controller calls required to reach each target is estimated using the continuous latent predictor.

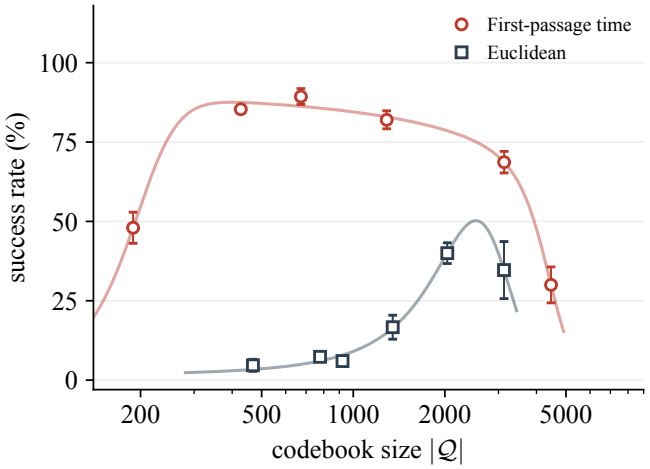


Fig. 5: State vector quantization with different codebook sizes.

VI. Limitations and Future Work

VQ-WM provides a foundation for further improvements in planning efficiency and task generality. First, the computational efficiency of its discrete search makes frequent MPC-style replanning practical. Our experiments use a single plan per episode to evaluate the reliability of our feedback motion planning framework, leaving the benefits of replanning unexplored. Future work can investigate how replanning frequency affects computational cost and recovery from deviations that require revising the planned sequence. Second, VQ-WM's modular design offers a direct path toward multi-task settings. A pretrained vision-language model can provide instruction-conditioned representations, while a multi-task policy can serve as the controller. Controller-aware vector quantization can then adapt the planning space to the selected controller with planning process unchanged. Evaluating these combinations across tasks is a promising direction for extending VQ-WM's applicability.

VII. Conclusion

We presented VQ-WM, a latent feedback motion planning framework that integrates discrete long-horizon search with a continuous controller for closed-loop execution. First-passage time estimated under learned closed-loop dynamics connects these two roles: it defines controller-dependent coverage regions and uses them to construct vector-quantized planning states. Discrete search uses the discrete latent predictor to

plan over representative states, while feedback control executes the resulting targets. Real-world Tower of Hanoi experiments demonstrate the effectiveness of this integration of planning and control, with VQ-WM outperforming state-of-the-art VLA, WAM, and world foundation model baselines. Simulation benchmarks spanning manipulation, and locomotion further demonstrate its broader applicability. These results support controller-dependent state abstraction as a practical foundation for long-horizon planning and reliable closed-loop execution.

REFERENCES

[1] G. Zhou, H. Pan, Y. LeCun, and L. Pinto, "Dino-wm: World models on pre-trained visual features enable zero-shot planning," 2024. [Online]. Available: https://arxiv.org/abs/2411.04983

[2] L. Maes, Q. Le Lidec, D. Scieur, Y. LeCun, and R. Balestriero, "LeWorldModel: Stable end-to-end joint-embedding predictive architecture from pixels," *arXiv preprint arXiv:2603.19312*, 2026. [Online]. Available: https://arxiv.org/abs/2603.19312

[3] W. Zhang, B. Terver, M. Rabbat, Y. LeCun, and R. Balestriero, "H-JEPA: End-to-end learning of hierarchical world models for visual planning," *arXiv preprint arXiv:2610.06805*, 2026. [Online]. Available: https://arxiv.org/abs/2610.06805

[4] D. Hafner, K.-H. Lee, I. Fischer, and P. Abbeel, "Deep hierarchical planning from pixels," *Advances in Neural Information Processing Systems*, vol. 35, pp. 26 091–26 104, 2022.

[5] C. Gumbsch, N. Sajid, G. Martius, and M. V. Butz, "Learning hierarchical world models with adaptive temporal abstractions from discrete latent dynamics," in *International Conference on Learning Representations (ICLR)*, 2024.

[6] N. Caselli, F. Massafra, S. Punzo, S. Lo Sardo, I. Pantelidis, and S. K. Bhethanabhotla, "Mind the gap: Promises and pitfalls of hierarchical planning in LeWorldModel," *arXiv preprint arXiv:2607.12547*, 2026.

[7] M. Nazeri, A. Card, S. Huber, A. Pokhrel, Y. Wang, R. Hammele, D. Song, S. Pirk, and X. Xiao, "Hydra: A navigation world action model with discrete latent planning and continuous flow-matching execution," *arXiv preprint arXiv:2608.28995*, 2026.

[8] Y. Wang, O. Bounou, G. Zhou, R. Balestriero, T. G. Rudner, Y. LeCun, and M. Ren, "Temporal straightening for latent planning," *arXiv preprint arXiv:2603.12231*, 2026.

[9] R. R. Burridge, A. A. Rizzi, and D. E. Koditschek, "Sequential composition of dynamically dexterous robot behaviors," *The International Journal of Robotics Research*, vol. 18, no. 6, pp. 534–555, 1999.

[10] A. Majumdar and R. Tedrake, "Funnel libraries for real-time robust feedback motion planning," *The International Journal of Robotics Research*, vol. 36, no. 8, pp. 947–982, 2017.

[11] R. Tedrake, "LQR-trees: Feedback motion planning on sparse randomized trees," in *Proceedings of Robotics: Science and Systems*, Seattle, USA, June 2009.

[12] M. Watter, J. T. Springenberg, J. Boedecker, and M. Riedmiller, "Embed to control: A locally linear latent dynamics model for control from raw images," in *Advances in Neural Information Processing Systems*, vol. 28, 2015. [Online]. Available: https://arxiv.org/abs/1506.07365

[13] D. Hafner, T. Lillicrap, I. Fischer, R. Villegas, D. Ha, H. Lee, and J. Davidson, "Learning latent dynamics for planning from pixels," in *Proceedings of the 36th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 97, 2019, pp. 2555–2565. [Online]. Available: https://proceedings.mlr.press/v97/hafner19a.html

[14] F. Ebert, C. Finn, S. Dasari, A. Xie, A. Lee, and S. Levine, "Visual foresight: Model-based deep reinforcement learning for vision-based robotic control," *arXiv preprint arXiv:1812.00568*, 2018. [Online]. Available: https://arxiv.org/abs/1812.00568

[15] A. Srinivas, A. Jabri, P. Abbeel, S. Levine, and C. Finn, "Universal planning networks: Learning generalizable representations for visuomotor control," in *Proceedings of the 35th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 80, 2018, pp. 4732–4741. [Online]. Available: https://proceedings.mlr.press/v80/srinivas18b.html

[16] D. Hafner, T. Lillicrap, J. Ba, and M. Norouzi, "Dream to control: Learning behaviors by latent imagination," in *International Conference on Learning Representations*, 2020. [Online]. Available: https://openreview.net/forum?id=S1lOTC4tDS

[17] N. Hansen, X. Wang, and H. Su, "Temporal difference learning for model predictive control," in *Proceedings of the 39th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 162, 2022. [Online]. Available: https://proceedings.mlr.press/v162/hansen22a.html

[18] N. Hansen, H. Su, and X. Wang, "TD-MPC2: Scalable, robust world models for continuous control," in *International Conference on Learning Representations*, 2024. [Online]. Available: https://arxiv.org/abs/2310.16828

[19] U. Sobal, W. Zhang, K. Cho, R. Balestriero, T. G. J. Rudner, and Y. LeCun, "Learning from reward-free offline data: A case for planning with latent dynamics models," in *Advances in Neural Information Processing Systems*, vol. 38, 2025.

[20] M. Assran, A. Bardes, D. Fan, Q. Garrido, R. Howes, M. Komeili *et al.*, "V-JEPA 2: Self-supervised video models enable understanding, prediction and planning," *arXiv preprint arXiv:2506.09985*, 2025. [Online]. Available: https://arxiv.org/abs/2506.09985

[21] W. Zhang, B. Terver, A. Zholus, S. Chitnis, H. Sutaria, M. Assran, R. Balestriero, A. Bar, A. Bardes, Y. LeCun *et al.*, "Hierarchical planning with latent world models," *arXiv preprint arXiv:2604.03208*, 2026.

[22] B. Eysenbach, R. Salakhutdinov, and S. Levine, "Search on the replay buffer: Bridging planning and reinforcement learning," in *Advances in Neural Information Processing Systems*, vol. 32, 2019. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2019/hash/5c48ff18e0a47baaf81d8b8ea51eec92-Abstract.html

[23] L. Zhang, G. Yang, and B. C. Stadie, "World model as a graph: Learning latent landmarks for planning," in *Proceedings of the 38th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 139, 2021, pp. 12 611–12 620. [Online]. Available: https://proceedings.mlr.press/v139/zhang21x.html

[24] C. Lynch, M. Khansari, T. Xiao, V. Kumar, J. Tompson, S. Levine, and P. Sermanet, "Learning latent plans from play," in *Proceedings of the Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 100, 2020, pp. 1113–1132. [Online]. Available: https://proceedings.mlr.press/v100/lynch20a.html

[25] K. Paster, S. A. McIlraith, and J. Ba, "Planning from pixels using inverse dynamics models," in *International Conference on Learning Representations*, 2021. [Online]. Available: https://openreview.net/forum?id=V6BjBgku7Ro

[26] A. van den Oord, O. Vinyals, and K. Kavukcuoglu, "Neural discrete representation learning," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[27] L. Maes, Q. L. Lidec, L. Facury, N. Massaudi, A. Chaurasia, F. Capuano, R. Gao, T. Gillin, D. Haramati, D. Scieur *et al.*, "stable-worldmodel: A platform for reproducible world modeling research and evaluation," *arXiv preprint arXiv:2605.21800*, 2026.

[28] Trossen Robotics, "WidowX AI robotic arm," https://www.trossenrobotics.com/widowx-ai, accessed: October 6, 2026.

[29] H. K. Cheng, S. W. Oh, B. Price, J.-Y. Lee, and A. Schwing, "Putting the object back into video object segmentation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

[30] M. J. Kim, Y. Gao, T.-Y. Lin, Y.-C. Lin, Y. Ge, G. Lam, P. Liang, S. Song, M.-Y. Liu, C. Finn *et al.*, "Cosmos policy: Fine-tuning video models for visuomotor control and planning," *arXiv preprint arXiv:2601.16163*, 2026.

[31] K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail *et al.*, "$\pi_{0.5}$: a vision-language-action model with open-world generalization," in *Proceedings of The 9th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, J. Lim, S. Song, and H.-W. Park, Eds., vol. 305. PMLR, 27–30 Sep 2025, pp. 17–40. [Online]. Available: https://proceedings.mlr.press/v305/black25a.html

[32] R. Y. Rubinstein, "Optimization of computer simulation models with rare events," *European Journal of Operational Research*, vol. 99, no. 1, pp. 89–112, 1997. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0377221796003852

[33] E. Todorov, T. Erez, and Y. Tassa, "Mujoco: A physics engine for model-based control," in *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2012, pp. 5026–5033.